

AI BASED STUDENT PERFORMANCE PREDICTION AND PERSONALIZED LEARNING RECOMMENDATION

Complete Project Detail, Pages, Frontend and Backend Structure

Tech Stack: HTML, CSS, JavaScript, Node.js, Express, MySQL + Python ML
(scikit-learn, Flask)

Submitted by: _____ Roll No: _____

Guide: _____ Department: _____

Institution: _____ Year: _____

Table of Contents

1. Project Overview
2. Objectives and Scope
3. Users and Roles
4. Modules and Features
5. Complete Page List (All Pages)
6. Frontend Structure
7. Backend Structure
8. Database Design
9. API Endpoints
10. Dataset and ML Model (Kaggle)
11. Recommendation Engine (ML + Rules)
12. System Working Flow
13. Security
14. Testing Plan
15. Development Timeline
16. Future Scope

1. Project Overview

Ye ek web based system hai jo students ka academic data (marks, attendance, assignments, study habits) leke unki **performance predict** karta hai aur har student ko **personalized learning recommendation** deta hai. Isse teachers ko at-risk students jaldi pata chal jate hain aur students ko pata chalta hai ki kis subject par kitna focus karna hai.

Problem Statement

Traditional system me student ki performance exam ke baad hi pata chalti hai, tab tak sudharne ka time nikal chuka hota hai. Har student ke liye alag study plan bhi nahi hota.

Proposed Solution

- Kaggle dataset par trained ML model se performance score aur level (High / Medium / Low) predict karna.
- Weak subjects aur reasons (low attendance, kam study hours) pehchanna.
- Personalized study plan, tips aur resources dena.
- Teacher aur admin ke liye reports aur at-risk list.

2. Objectives and Scope

Objectives

- Student performance ka early prediction karna.
- Weak areas ki automatic identification.
- Personalized recommendation generate karna.
- Role based dashboards (Admin, Teacher, Student).
- Progress history aur graphs ke through improvement track karna.

Scope

School/college level students, subject-wise marks, attendance aur study habits par based prediction.

Limitation: prediction ki accuracy input data ki quality par depend karti hai.

3. Users and Roles

Role	Access / Responsibilities
Admin	Teachers, students, subjects, classes manage karna. Overall reports aur system statistics dekhna.
Teacher	Marks, assignments aur attendance enter/update karna. Class performance aur at-risk students dekhna.
Student	Apna dashboard, marks, study habits form, prediction, recommendations aur progress history dekhna.

4. Modules and Features

Module	Features
Authentication	Register, Login, Logout, JWT token, password hashing, role based access, forgot password.
Admin Management	Student/Teacher CRUD, subjects, classes, system reports, dashboard statistics.
Marks and Attendance	Exam marks, assignment scores, attendance percentage entry, update, delete.

Module	Features
Study Habits	Weekly study hours, sleep, tutoring, motivation, resources etc. ka form (ML model ke features).
Prediction	Student ka performance score aur level (High/Medium/Low) nikalna aur history save karna.
Recommendation	Weak subject tips, behaviour tips, resources, overall study plan.
Reports and Charts	Subject-wise bar chart, progress line chart, class distribution pie chart, export option.

5. Complete Page List (All Pages)

Neeche project ke saare pages aur unme kya-kya hoga, detail me diya hai.

5.1 Public Pages

Page	File	Content / Components
Landing Page	index.html	Navbar, hero section (project intro), features cards, how it works steps, Login/Register buttons, footer.
Login	login.html	Email, password, role select, Login button, forgot password link, error message area.
Register	register.html	Name, email, password, confirm password, role, class/roll no (student), Register button.

5.2 Admin Pages

Page	File	Content / Components
Admin Dashboard	admin/dashboard.html	Stat cards (total students, teachers, avg performance, at-risk count), class-wise performance chart, recent activity.
Manage Students	admin/manage-students.html	Search and filter, students table, Add/Edit/Delete buttons, modal form (name, class, roll no, email).
Manage Teachers	admin/manage-teachers.html	Teachers table, add/edit/delete, assign subjects and classes.
Manage Subjects	admin/manage-subjects.html	Subjects list, add/edit subject, assign to class.
Reports	admin/reports.html	Filter by class/subject/date, performance distribution pie chart, top performers, at-risk list, export CSV/PDF.

5.3 Teacher Pages

Page	File	Content / Components
Teacher Dashboard	teacher/dashboard.html	Assigned classes, class average, at-risk count, quick links.
My Students	teacher/students.html	Class-wise student list, view profile, view prediction button.
Add Marks	teacher/add-marks.html	Select class, subject, exam type; table to enter exam and assignment marks; Save button.
Attendance	teacher/attendance.html	Date select, student list with present/absent toggle, monthly attendance percentage.
At-Risk Students	teacher/at-risk.html	List of Low performance students with reasons, suggested action, view recommendation.

5.4 Student Pages

Page	File	Content / Components
Student Dashboard	student/dashboard.html	Performance level card, score, attendance, subject-wise bar chart, latest recommendation summary.
Profile	student/profile.html	Personal details, class, roll no, change password.

Page	File	Content / Components
My Marks	student/my-marks.html	Subject-wise marks table (exam, assignment), total, percentage, trend chart.
Study Habits	student/study-habits.html	Form: weekly study hours, sleep, tutoring, physical activity, motivation, internet access (ML features); Save button.
Prediction	student/prediction.html	'Predict My Performance' button, result card (level + score), factor breakdown, gauge/progress chart.
Recommendations	student/recommendations.html	Overall study plan, weak subjects with priority badges, tips, resource links, behaviour suggestions.
History	student/history.html	Past predictions table, line chart of score over time to show improvement.

5.5 Common Components (har page me)

- **Navbar:** logo, user name, logout.
- **Sidebar:** role ke hisaab se menu links.
- **Alerts/Toasts:** success aur error messages.
- **Footer:** copyright.
- **Loader:** API call ke time spinner.

6. Frontend Structure

```
frontend/
|-- index.html
|-- login.html
|-- register.html
|
|-- admin/
|   |-- dashboard.html
|   |-- manage-students.html
|   |-- manage-teachers.html
|   |-- manage-subjects.html
|   +-- reports.html
|
|-- teacher/
|   |-- dashboard.html
|   |-- students.html
|   |-- add-marks.html
|   |-- attendance.html
|   +-- at-risk.html
|
|-- student/
|   |-- dashboard.html
|   |-- profile.html
|   |-- my-marks.html
|   |-- study-habits.html
|   |-- prediction.html
|   |-- recommendations.html
|   +-- history.html
|
|-- css/
|   |-- style.css          (common styles, variables, buttons, cards)
|   |-- auth.css          (login/register)
|   |-- dashboard.css     (sidebar, navbar, stat cards, tables)
|   +-- responsive.css    (mobile/tablet media queries)
|
|-- js/
|   |-- config.js         (API base URL)
|   |-- api.js            (fetch helper, token attach)
|   |-- auth.js           (login, register, logout)
|   |-- guard.js          (role check, redirect if not logged in)
|   |-- admin.js
|   |-- teacher.js
|   |-- student.js
|   |-- charts.js         (Chart.js graphs)
|   +-- utils.js          (validation, alerts, date format)
|
+-- assets/
    |-- images/
    +-- icons/
```

Frontend Working

- Login ke baad JWT token **localStorage** me save hota hai.
- **guard.js** har page par token aur role check karta hai.
- **api.js** me fetch() ke saath Authorization header automatic lagta hai.
- **charts.js** backend ke data se Chart.js graphs banata hai.
- **responsive.css** se site mobile par bhi sahi dikhti hai.

7. Backend Structure

```
backend/
|-- server.js             (server start)
|-- app.js               (express setup, middleware, routes)
|-- .env                 (PORT, DB credentials, JWT_SECRET)
|-- package.json
```

```

|
|-- config/
|   +-- db.js                (MySQL connection pool)
|
|-- routes/
|   |-- authRoutes.js
|   |-- adminRoutes.js
|   |-- teacherRoutes.js
|   |-- studentRoutes.js
|   |-- marksRoutes.js
|   |-- attendanceRoutes.js
|   |-- predictionRoutes.js
|   +-- recommendationRoutes.js
|
|-- controllers/
|   |-- authController.js
|   |-- adminController.js
|   |-- teacherController.js
|   |-- studentController.js
|   |-- marksController.js
|   |-- attendanceController.js
|   |-- predictionController.js
|   +-- recommendationController.js
|
|-- models/                  (SQL queries)
|   |-- userModel.js
|   |-- studentModel.js
|   |-- marksModel.js
|   +-- predictionModel.js
|
|-- middleware/
|   |-- authMiddleware.js    (JWT verify)
|   |-- roleMiddleware.js    (admin/teacher/student check)
|   |-- validate.js          (input validation)
|   +-- errorHandler.js
|
|-- services/
|   +-- mlClient.js          (Python ML service ko HTTP call)
|
|-- utils/
|   |-- hash.js              (bcrypt)
|   +-- token.js             (JWT sign/verify)
|
+-- database/
    |-- schema.sql
    +-- seed.sql             (dummy data)

```

Packages

express, mysql2, cors, dotenv, bcryptjs, jsonwebtoken, express-validator, nodemon (dev). Frontend me Chart.js.

7.1 ML Service Structure (Python)

Model Python me train hota hai aur ek chhoti Flask API ke through Node.js backend se connect hota hai.

```

ml-service/
|-- data/
|   +-- StudentPerformanceFactors.csv    (Kaggle se download)
|-- notebooks/
|   |-- 01_EDA.ipynb
|   +-- 02_model_experiments.ipynb
|-- models/
|   |-- classifier.pkl    (Low/Medium/High)
|   |-- regressor.pkl    (Exam score)
|   +-- meta.json        (features, thresholds, metrics)
|-- reports/
|   |-- confusion_matrix.png
|   +-- feature_importance.png
|-- train_model.py
|-- recommender.py

```

```
|-- app.py          (Flask API, port 5001)
+-- requirements.txt (pandas, scikit-learn, flask, joblib, matplotlib)
```

8. Database Design (MySQL)

Table	Columns
users	id (PK), name, email (unique), password, role (admin/teacher/student), created_at
classes	id (PK), class_name, section
students	id (PK), user_id (FK), class_id (FK), roll_no
teachers	id (PK), user_id (FK), department
subjects	id (PK), subject_name, class_id (FK), teacher_id (FK)
marks	id (PK), student_id (FK), subject_id (FK), exam_score, assignment_score, exam_type, date
attendance	id (PK), student_id (FK), date, status (present/absent)
study_habits	id (PK), student_id (FK), hours_studied_weekly, sleep_hours, tutoring_sessions, physical_activity, extracurricular, motivation_level, parental_involvement, access_to_resources, internet_access, family_income, teacher_quality, updated_at
predictions	id (PK), student_id (FK), predicted_level, predicted_score, probabilities (JSON), model_version, created_at
recommendations	id (PK), student_id (FK), subject_id (FK), type, priority, tip, created_at

Relations

- users 1:1 students / teachers
- classes 1:N students
- students 1:N marks, attendance, predictions, recommendations
- subjects 1:N marks

Sample SQL (core tables)

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  role ENUM('admin','teacher','student') NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE students (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  class_id INT,  
  roll_no VARCHAR(20),  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (class_id) REFERENCES classes(id)  
);  
  
CREATE TABLE marks (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  student_id INT NOT NULL,  
  subject_id INT NOT NULL,  
  exam_score DECIMAL(5,2),  
  assignment_score DECIMAL(5,2),  
  exam_type VARCHAR(30),  
  date DATE,  
  FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,  
  FOREIGN KEY (subject_id) REFERENCES subjects(id)  
);  
  
CREATE TABLE predictions (  

```

```

id INT AUTO_INCREMENT PRIMARY KEY,
student_id INT NOT NULL,
predicted_level ENUM('High', 'Medium', 'Low'),
predicted_score DECIMAL(5,2),
probabilities JSON,
model_version VARCHAR(20),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE
);

```

9. API Endpoints

Method	Endpoint	Role	Purpose
POST	/api/auth/register	Public	Naya user register
POST	/api/auth/login	Public	Login, JWT token return
GET/POST/PUT/DELETE	/api/admin/students	Admin	Students CRUD
GET/POST/PUT/DELETE	/api/admin/teachers	Admin	Teachers CRUD
GET/POST	/api/admin/subjects	Admin	Subjects manage
GET	/api/admin/reports	Admin	Overall reports
GET	/api/teacher/students	Teacher	Class students list
POST/PUT	/api/marks	Teacher	Marks add/update
POST	/api/attendance	Teacher	Attendance save
GET	/api/teacher/at-risk	Teacher	Low performance students
GET	/api/student/profile	Student	Profile details
GET	/api/student/marks	Student	Apne marks
POST	/api/student/study-habits	Student	Study habits save
GET	/api/predict/:studentId	Student/Teacher	Prediction + recommendations
GET	/api/predict/history/:studentId	Student/Teacher	Past predictions
GET	/api/recommendations/:studentId	Student/Teacher	Saved recommendations

10. Dataset and ML Model (Kaggle)

Yahan prediction ke liye formula nahi, balki Kaggle ke real dataset par **trained Machine Learning model** use hoga. Model Python (scikit-learn) me train hoga, aur Node.js backend usse Flask API ke through call karega.

10.1 Dataset Selection

Dataset (Kaggle)	Size	Use
Student Performance Factors	about 6,600 rows	Main dataset (recommended). Hours_Studied, Attendance, Sleep_Hours, Previous_Scores, Tutoring_Sessions, Motivation_Level etc. hamare app ke inputs se match karte hain. Target: Exam_Score.
Student Performance Data Set (UCI, student-mat.csv)	395 rows	Optional. G1, G2, G3, studytime, failures, absences. Chhota hai, comparison ya extra experiment ke liye.
Students Performance in Exams	1,000 rows	Optional. Math, reading, writing scores, subject-wise analysis ke liye.

Note: Download ke baad `df.columns` print karke column names zarur check kar lena, kyunki dataset versions me naam thoda alag ho sakte hain.

10.2 Features aur App se Mapping

Feature	Type	App me kahan se aayega
Hours_Studied	Number (weekly)	Study habits form
Attendance	Number (%)	attendance table se calculate
Sleep_Hours	Number	Study habits form
Previous_Scores	Number (%)	marks table ka average
Tutoring_Sessions	Number	Study habits form
Physical_Activity	Number (hrs/week)	Study habits form
Extracurricular_Activities	Yes / No	Study habits form
Motivation_Level	Low / Medium / High	Student self rating
Parental_Involvement	Low / Medium / High	Student/teacher input
Access_to_Resources	Low / Medium / High	Study habits form
Internet_Access	Yes / No	Study habits form
Family_Income	Low / Medium / High	Admin/student profile (optional)
Teacher_Quality	Low / Medium / High	Admin rating (optional)
Exam_Score (target)	Number	Regression: predicted score
Level (target)	Low / Medium / High	Classification: Exam_Score se banega

Level banane ke liye training data ke Exam_Score ke 33rd aur 66th percentile thresholds nikalte hain: neeche wale Low, beech wale Medium, upar wale High. Thresholds me `thresholds.json` me save hote hain.

10.3 ML Pipeline (Step by Step)

Step	Kya karna hai
1. Load	CSV padho, duplicates hatao.
2. Clean	Missing values (median / most frequent), outliers (Exam_Score > 100) check.
3. EDA	Distribution plots, correlation heatmap, study hours vs score, attendance vs score. Report ke liye figures save karo.
4. Encode and Scale	Categorical columns One-Hot, numeric columns StandardScaler (ColumnTransformer).
5. Split	80% train, 20% test (stratified).
6. Compare Models	5-fold cross validation se multiple models compare.
7. Tune	Best model par RandomizedSearchCV / GridSearchCV.
8. Evaluate	Test set par metrics, confusion matrix, feature importance.
9. Save	joblib se .pkl aur meta.json.
10. Serve	Flask API se Node.js backend ko connect.

10.4 Models aur Metrics

Task	Models to compare	Metrics
Classification (Low/Medium/High)	Logistic Regression (baseline), Random Forest, Gradient Boosting, XGBoost (optional)	Accuracy, Precision, Recall, F1 (macro), Confusion Matrix
Regression (Exam Score)	Linear Regression, Random Forest Regressor, Gradient Boosting Regressor	MAE, RMSE, R2 score

Result table (training ke baad apne actual numbers bharna):

Model	CV F1 (macro)	Test Accuracy	Test F1	Remark
Logistic Regression	_____	_____	_____	Baseline
Random Forest	_____	_____	_____	
Gradient Boosting	_____	_____	_____	Best model choose karo
XGBoost (optional)	_____	_____	_____	

10.5 Training Code (ml-service/train_model.py)

```
import json, joblib, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import (RandomForestClassifier, GradientBoostingClassifier,
                              GradientBoostingRegressor)
from sklearn.metrics import (classification_report, confusion_matrix, accuracy_score,
                              f1_score, mean_absolute_error, mean_squared_error, r2_score)
from sklearn.inspection import permutation_importance

df = pd.read_csv("data/StudentPerformanceFactors.csv").drop_duplicates()
df = df[df["Exam_Score"] <= 100]

NUM = ["Hours_Studied", "Attendance", "Sleep_Hours", "Previous_Scores",
       "Tutoring_Sessions", "Physical_Activity"]
CAT = ["Extracurricular_Activities", "Motivation_Level", "Parental_Involvement",
       "Access_to_Resources", "Internet_Access", "Family_Income", "Teacher_Quality"]
FEATURES = NUM + CAT

q1, q2 = np.percentile(df["Exam_Score"], [33, 66])
df["Level"] = pd.cut(df["Exam_Score"], [-np.inf, q1, q2, np.inf],
                    labels=["Low", "Medium", "High"])

X, y_c, y_r = df[FEATURES], df["Level"], df["Exam_Score"]
Xtr, Xte, yc_tr, yc_te, yr_tr, yr_te = train_test_split(
    X, y_c, y_r, test_size=0.2, random_state=42, stratify=y_c)

def make_pre():
    num = Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())])
    cat = Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                    ("oh", OneHotEncoder(handle_unknown="ignore"))])
    return ColumnTransformer([("num", num, NUM), ("cat", cat, CAT)])

models = {
    "LogisticRegression": LogisticRegression(max_iter=1000),
    "RandomForest": RandomForestClassifier(n_estimators=300, random_state=42),
    "GradientBoosting": GradientBoostingClassifier(random_state=42),
}
best_name, best_f1 = None, 0
for name, m in models.items():
    pipe = Pipeline([("pre", make_pre()), ("model", m)])
    f1 = cross_val_score(pipe, Xtr, yc_tr, cv=5, scoring="f1_macro").mean()
    print(f"{name}: CV F1 = {f1:.4f}")
    if f1 > best_f1:
        best_name, best_f1 = name, f1

clf = Pipeline([("pre", make_pre()), ("model", models[best_name])]).fit(Xtr, yc_tr)
pred = clf.predict(Xte)
print("Best:", best_name)
print(classification_report(yc_te, pred))
print(confusion_matrix(yc_te, pred, labels=["Low", "Medium", "High"]))

reg = Pipeline([("pre", make_pre()),
                ("model", GradientBoostingRegressor(random_state=42))]).fit(Xtr, yr_tr)
rp = reg.predict(Xte)
mae = mean_absolute_error(yr_te, rp)
rmse = float(np.sqrt(mean_squared_error(yr_te, rp)))
r2 = r2_score(yr_te, rp)
print(f"MAE={mae:.2f} RMSE={rmse:.2f} R2={r2:.3f}")

imp = permutation_importance(clf, Xte, yc_te, n_repeats=5, random_state=42)
pd.Series(imp.importances_mean, index=FEATURES).sort_values().plot.barh()
plt.tight_layout(); plt.savefig("reports/feature_importance.png")

joblib.dump(clf, "models/classifier.pkl")
```

```

joblib.dump(reg, "models/regressor.pkl")
json.dump({"version": "1.0", "features": FEATURES, "num": NUM, "cat": CAT,
          "thresholds": [float(q1), float(q2)], "best_model": best_name,
          "accuracy": float(accuracy_score(yc_te, pred)),
          "f1_macro": float(f1_score(yc_te, pred, average="macro")),
          "mae": float(mae), "rmse": rmse, "r2": float(r2)},
          open("models/meta.json", "w"), indent=2)

```

10.6 Flask API (ml-service/app.py)

```

from flask import Flask, request, jsonify
import joblib, json, pandas as pd
from recommender import what_if, subject_rules, PLANS

app = Flask(__name__)
clf = joblib.load("models/classifier.pkl")
reg = joblib.load("models/regressor.pkl")
meta = json.load(open("models/meta.json"))
F = meta["features"]

@app.post("/predict")
def predict():
    data = request.get_json()
    X = pd.DataFrame([data])[F]
    level = str(clf.predict(X)[0])
    proba = {str(c): round(float(p), 3)
             for c, p in zip(clf.classes_, clf.predict_proba(X)[0])}
    score, tips = what_if(reg, data, F)
    return jsonify({
        "level": level,
        "probabilities": proba,
        "predicted_score": round(score, 1),
        "model_version": meta["version"],
        "overall_plan": PLANS[level],
        "recommendations": tips,
        "subject_tips": subject_rules(data.get("subjects", [])),
    })

if __name__ == "__main__":
    app.run(port=5001)

```

10.7 Node.js Integration

services/mlClient.js (Node 18+ me fetch built-in hai):

```

const ML_URL = process.env.ML_URL || "http://localhost:5001";

async function getPrediction(features) {
    const res = await fetch(`${ML_URL}/predict`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(features),
    });
    if (!res.ok) throw new Error("ML service error");
    return res.json();
}

module.exports = { getPrediction };

```

controllers/predictionController.js:

```

const { getPrediction } = require("../services/mlClient");

exports.predict = async (req, res, next) => {
    try {
        const id = req.params.studentId;
        const features = await studentModel.getMLFeatures(id); // marks + attendance + habits
        const result = await getPrediction(features);
        await predictionModel.save(id, result); // predictions table
        res.json(result);
    } catch (err) { next(err); }
};

```

10.8 Prediction Page par kya dikhega

- Predicted level (color coded) aur predicted exam score.
- Confidence: Low / Medium / High ki probability bars.
- Previous prediction se comparison (improved ya declined).
- 'View Recommendations' button.

10.9 Report ke liye Figures

- Dataset distribution aur correlation heatmap.
- Model comparison bar chart (CV F1).
- Confusion matrix.
- Feature importance chart (kaunsa factor score ko sabse zyada affect karta hai).
- Actual vs predicted score scatter plot.

11. Recommendation Engine (ML + Rules)

Recommendation engine **hybrid** hai: ML model se *what-if analysis* (kaunsi aadat sudharne se score sabse zyada badhega) aur rules se subject-wise tips.

11.1 Recommendation Layers

Layer	Kaise kaam karta hai	Output
ML What-If	Student ke features me ek-ek change karke regressor se naya score nikalte hain. Jis change se zyada gain mile wo upar.	Top tips with expected score gain
Feature Importance	Permutation importance se pata chalta hai dataset me kaunsa factor sabse important hai.	Report/Admin insights
Subject Rules	Marks table ke subject-wise marks se weak subjects pehchanna (dataset me subject-wise data nahi hota).	Weak subject tips and resources
Overall Plan	Predicted level ke hisaab se study plan.	Plan text

11.2 What-If Examples

Change	Example Tip	Expected Gain
Hours_Studied +5	Weekly study hours 5 ghante badhao	Model se calculate hoga
Attendance +10	Attendance 10% badhao	Model se calculate hoga
Sleep_Hours minimum 7	Roz kam se kam 7 ghante neend lo	Model se calculate hoga
Tutoring_Sessions +2	2 extra tutoring/doubt sessions lo	Model se calculate hoga
Physical_Activity +1	Hafte me thodi physical activity badhao	Model se calculate hoga

11.3 Code (ml-service/recommender.py)

```
import pandas as pd

PLANS = {
    "Low": "Basics se start karo, daily timetable follow karo, teacher se doubt session lo.",
    "Medium": "Weekly revision aur mock tests karo, weak topics par focus rakho.",
    "High": "Advanced problems, competitive exams aur projects try karo.",
}

ACTIONS = [
    {"feature": "Hours_Studied", "mode": "add", "value": 5, "max": 44,
     "tip": "Weekly study hours 5 ghante badhao"},
    {"feature": "Attendance", "mode": "add", "value": 10, "max": 100,
     "tip": "Attendance 10% badhao"},
    {"feature": "Sleep_Hours", "mode": "min", "value": 7,
     "tip": "Roz kam se kam 7 ghante neend lo"},
    {"feature": "Tutoring_Sessions", "mode": "add", "value": 2, "max": 8,
     "tip": "2 extra tutoring/doubt sessions lo"},
    {"feature": "Physical_Activity", "mode": "add", "value": 1, "max": 6,
     "tip": "Physical activity thodi badhao"},
]

def apply(v, a):
    if a["mode"] == "add":
        return min(v + a["value"], a["max"])
    return max(v, a["value"])

def what_if(reg, row, features):
    base = float(reg.predict(pd.DataFrame([row])[features])[0])
    out = []
```

```

for a in ACTIONS:
    new_row = dict(row)
    new_row[a["feature"]] = apply(row[a["feature"]], a)
    new = float(reg.predict(pd.DataFrame([new_row])[features])[0])
    gain = new - base
    if gain >= 0.3:
        out.append({"feature": a["feature"], "tip": a["tip"],
                    "expected_gain": round(gain, 1)})
    out.sort(key=lambda x: -x["expected_gain"])
    return base, out

def subject_rules(subjects):
    tips = []
    for s in subjects:
        # [{"name": "Maths", "marks": 42}, ...]
        if s["marks"] < 50:
            tips.append({"subject": s["name"], "priority": "High",
                        "tip": f"{s['name']} me {s['marks']}% hai. Roz 1 ghanta do."})
        elif s["marks"] < 70:
            tips.append({"subject": s["name"], "priority": "Medium",
                        "tip": f"{s['name']} me weekly test lo aur mistakes note karo."})
    return tips

```

11.4 Sample API Output

```

{
  "level": "Medium",
  "probabilities": { "High": 0.18, "Low": 0.21, "Medium": 0.61 },
  "predicted_score": 66.4,
  "model_version": "1.0",
  "overall_plan": "Weekly revision aur mock tests karo, weak topics par focus rakho.",
  "recommendations": [
    { "feature": "Attendance", "tip": "Attendance 10% badhao", "expected_gain": 1.9 },
    { "feature": "Hours_Studied", "tip": "Study hours 5 badhao", "expected_gain": 1.4 }
  ],
  "subject_tips": [
    { "subject": "Maths", "priority": "High", "tip": "Maths me 42% hai. Roz 1 ghanta do..." }
  ]
}

```

(Upar ke numbers sirf example hain. Actual values trained model se aayengi.)

11.5 Recommendation Page par kya dikhega

- Overall study plan card.
- 'Aapke score ko sabse zyada badhane wale kadam' list, expected gain ke saath.
- Weak subjects list (red = High, orange = Medium priority) with resource links.
- Prediction history ke saath improvement track.

12. System Working Flow

Step	Action
1	Admin teachers aur students ko system me add karta hai.
2	Teacher marks, assignments aur attendance enter karta hai.
3	Student study habits form bharta hai.
4	Student 'Predict' button dabata hai (GET /api/predict/:id).
5	Backend fetches MySQL se fetch karke ML service (Flask) ko bhejta hai.
6	Trained model level, probability aur predicted score deta hai.
7	recommender.py what-if analysis se best improvement tips nikalta hai.
8	Result predictions table me save hota hai.
9	Frontend result, charts aur recommendations dikhata hai.
10	Teacher at-risk students ki list dekhkar action leta hai.

Architecture

```
Browser (HTML/CSS/JS + Chart.js)
  | fetch() with JWT
  v
Node.js + Express API
  |- Auth + Role Middleware
  |- Controllers
  |- services/mlClient.js --HTTP--> Python ML Service (Flask, port 5001)
                                   |- classifier.pkl (Level)
                                   |- regressor.pkl (Score)
                                   +- recommender.py (Tips)
  v
MySQL Database
```

13. Security

- Password **bcrypt** se hash.
- **JWT** token authentication.
- Role based route protection (admin/teacher/student).
- Parameterized queries se SQL injection se bachav.
- Input validation (express-validator).
- CORS configure, .env me secrets.
- ML service sirf Node backend se access ho (internal port), public na ho.

14. Testing Plan

Type	What to Test	Tool
Model	Test set metrics, overfitting check (train vs test), different inputs par prediction	scikit-learn, pytest
API	Saare endpoints, auth aur role access	Postman
Integration	Frontend to backend to DB full flow	Browser
Validation	Wrong input, empty fields, invalid token	Manual
Usability	Mobile aur desktop responsive check	Browser DevTools

Sample Test Cases

Input	Expected Output
Hours 35, Attendance 95, Sleep 8, Previous 85	High (expected)
Hours 20, Attendance 80, Sleep 7, Previous 65	Medium (expected)
Hours 5, Attendance 55, Sleep 5, Previous 45	Low + attendance/study tips
ML service band ho	Node friendly error deta hai, app crash nahi hota
Student role se /api/admin/students	403 Forbidden
Bina token ke /api/predict/1	401 Unauthorized

15. Development Timeline (5 Weeks)

Week	Work
Week 1	Kaggle dataset download, EDA, cleaning, preprocessing
Week 2	Model training, comparison, tuning, evaluation, Flask API aur recommender.py
Week 3	Database design, Node backend, auth, admin/teacher modules, marks and attendance CRUD
Week 4	Node to ML integration, student pages, prediction and recommendation pages, charts
Week 5	Testing, report figures, final report aur PPT

16. Future Scope

- Real college data par model retraining, XGBoost/deep learning aur SHAP explainability.
- Parent login aur email/SMS alerts.
- AI chatbot se doubt solving.
- Mobile app version.
- Learning management system (LMS) integration.
- Subject-wise video/quiz recommendations.